

CS253: Software Development & Operations

Assignment 1 Report

Memory-Efficient Versioned File Indexer

Chandradip Karmakar

Roll No: 240296

March 10, 2026

Indian Institute of Technology Kanpur

§ 1 Introduction and Project Goals

The main idea behind this project is to build a text-file indexing tool that can work on very large files without running out of memory. In real-world systems, log files can grow to enormous sizes, and loading them all at once would be slow and wasteful. To avoid this, the program reads files in small, fixed-size chunks, so only a tiny piece of the file sits in memory at any given moment.

On top of that, the system supports versioned file management. This means you can keep track of different copies (versions) of a log file and compare them with each other. The two main things users can do are: look up how many times a particular word appears, and ask for the top-K most frequent words in a given version.

§ 2 System Design and Object-Oriented Structure

The codebase is split into well-defined, independent pieces so that each part handles one job and one job only. This makes it easier to test, extend, and maintain.

2.1 Core Components

- **StatusLogger<T>** — A small templated utility class with a static `log()` method. It prints labelled diagnostic messages to the console and demonstrates the use of C++ templates in the codebase.
- **Indexer** — The central workhorse of the system. It handles both buffered file reading and word-frequency indexing in one place. It opens the file with `fopen`, reads it chunk by chunk using `fread`, and builds an `unordered_map` mapping each lowercase word to its frequency count. The buffer size (in KB) is passed in at runtime, so the user fully controls how much memory is used at any one time.
- **BaseQuery** — An abstract base class that declares a pure virtual `execute()` method. All query types inherit from this class, giving the system a uniform interface for running any kind of analysis.
- **SingleWordQuery** — A concrete subclass of **BaseQuery**. Given a version name and a target word, it prints how many times that word appears in the indexed file.
- **TopKQuery** — Another concrete subclass of **BaseQuery**. It sorts all words by frequency in descending order and prints the top-K most common ones.

- **Controller** — Ties everything together using function overloading. One `perform()` overload handles single-file queries (word lookup and top-K), while a second overload handles cross-version difference queries by comparing counts from two separate `Indexer` objects.

2.2 C++ Language Features Used

Table 1 summarises the mandatory C++ features and how each one appears in the actual code.

Feature	Mechanism	How It Is Used
Polymorphism	Dynamic Dispatch	<code>execute()</code> in <code>BaseQuery</code> is a pure virtual function, overridden by both <code>SingleWordQuery</code> and <code>TopKQuery</code> .
Templates	Generic Classes	<code>StatusLogger<T></code> is a templated class whose <code>log()</code> method works with any printable type <code>T</code> .
Overloading	Function Overloading	<code>Controller::perform()</code> has two overloads: one takes a single <code>Indexer</code> for word/top queries; the other takes two <code>Indexer</code> objects for cross-version diff queries.
Exceptions	Error Handling	<code>try</code> , <code>catch</code> , and <code>throw</code> are used throughout <code>main()</code> and <code>Indexer::process()</code> to catch problems such as a missing <code>--query</code> flag or an unreadable input file.

Table 1: C++ features and how they are realised in the codebase.

§ 3 How Bounded Memory Usage Works

Keeping memory usage low is the key design constraint of the whole project. Here is how it is achieved in practice.

1. **Fixed Buffer Size.** The program allocates a read buffer whose size is between 256 KB and 1024 KB — a range chosen by the user. No matter how large the input file is, this buffer never grows beyond 1024 KB.
2. **Split-Token Handling.** Sometimes a word falls right on the boundary between two consecutive buffers and gets cut in half. The Tokenizer keeps track of this partial word and joins it with the beginning of the next chunk, so every token is recovered correctly.
3. **Scaling Behaviour.** Once reading is done, the only memory that grows is the index itself — one entry per unique word per version. Processing memory is constant regardless of file size; index memory grows only with vocabulary size.

§ 4 How to Build and Run the Program

4.1 Compiling the Source

Use `g++` with the `-O2` optimisation flag:

Command:

```
g++ -O2 240296_Chandradip.cpp -o analyzer
```

4.2 Command-Line Flags

Table 2 lists every flag the `analyzer` binary accepts.

Flag	Description	Required For
<code>--file</code>	Path to the primary input file	Word and Top-K queries
<code>--file1/--file2</code>	Paths to the two files being compared	Difference queries
<code>--buffer</code>	Buffer size in KB (256–1024)	Each and every query type
<code>--query</code>	Query type: <code>word</code> , <code>top</code> , or <code>diff</code>	Each and every query type

Table 2: Command-line flags supported by the `analyzer` binary.

§ 5 Performance Testing and Results

This section shows how the program performs on real log files across three different types of query.

5.1 Experiment A: Single-Word Frequency Lookup

Here we ask how many times a specific word appears in a file. Two runs are shown below — one searching for the word `error` and one for `info` — both using a 512 KB buffer.

Command:

```
./analyzer --file test_logs.txt --version v1 --buffer 512 \
  --query word --word error
```

Output:

```
Version : v1
Count   : 605079
Buffer Size (KB): 512
Execution Time: 1.43547
```

Command:

```
./analyzer --file verbose_logs.txt --version v2 --buffer 512 \  
--query word --word info
```

Output:

```
Version : v2  
Count   : 109900  
Buffer Size (KB): 512  
Execution Time: 1.88402
```

5.2 Experiment B: Top-K Word Ranking

This experiment retrieves the most frequent words in a version. Two runs demonstrate how different buffer sizes affect execution time.

Command:

```
./analyzer --file test_logs.txt --version v1 --buffer 1024 \  
--query top --top 10
```

Output:

```
Top-10 words in version v1:  
devops    1209558  
debug     605150  
error     605079  
info      604266  
warning   604149  
orderservice 484437  
paymentservice 484078  
authservice 483842  
searchservice 483162  
userservice 483125  
Buffer Size (KB): 1024  
Execution Time: 1.62909
```

Command:

```
./analyzer --file test_logs.txt --version v1 --buffer 256 \  
--query top --top 10
```

Output:

```
Top-10 words in version v1:  
devops    1209558  
debug     605150
```

```
error      605079
info       604266
warning    604149
orderservice 484437
paymentservice 484078
authservice 483842
searchservice 483162
userservice 483125
Buffer Size (KB): 256
Execution Time : 2.60608
```

As expected, using a smaller buffer means more read operations are needed, which increases execution time from about 1.63 s to 2.61 s.

5.3 Experiment C: Inter-Version Frequency Comparison

Here we compare how often a word appears across two different versions of a log file. The output is the difference ($v_2 - v_1$), so a negative number means the word is less common in v_2 .

Command:

```
./analyzer --file1 test_logs.txt --version1 v1 \
--file2 verbose_logs.txt --version2 v2 \
--buffer 256 --query diff --word request
```

Output:

```
Difference (v2-v1): -247648
Buffer Size (KB) : 256
Execution Time : 6.72606
```

Command:

```
./analyzer --file1 test_logs.txt --version1 v1 \
--file2 verbose_logs.txt --version2 v2 \
--buffer 256 --query diff --word error
```

Output:

```
Difference (v2-v1): -495377
Buffer Size (KB) : 256
Execution Time : 3.74661
```

Both results show that the verbose logs file (v_2) contains far fewer occurrences of these words compared to test_logs (v_1). The diff query takes the longest because it must process two files sequentially.

§ 6 Summary and Closing Remarks

This project shows that it is possible to build a fast, useful text analysis tool without ever needing to load a whole file into memory. By reading in fixed-size chunks, handling split tokens carefully, and using clean object-oriented design, the program scales to arbitrarily large log files while staying within tight memory limits. The experiments confirm that the system works correctly across all three query types, and the results are consistent regardless of which buffer size is chosen — smaller buffers just take a bit longer to finish.